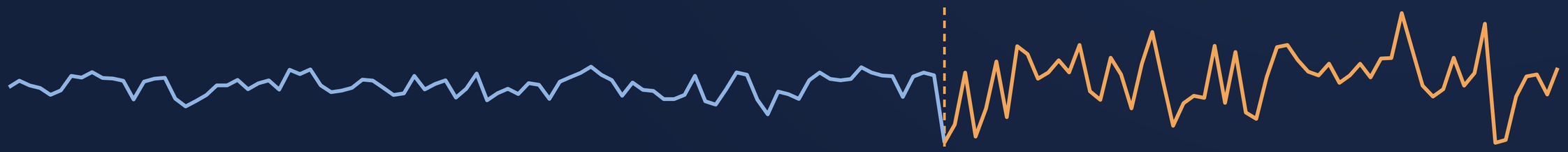
KU SBB · TEAM BLITZICT

Spotting when a pattern changes

Four attempts at the same problem: what each one did, why it scored better, and how I had to nerf my own best model to follow the rules



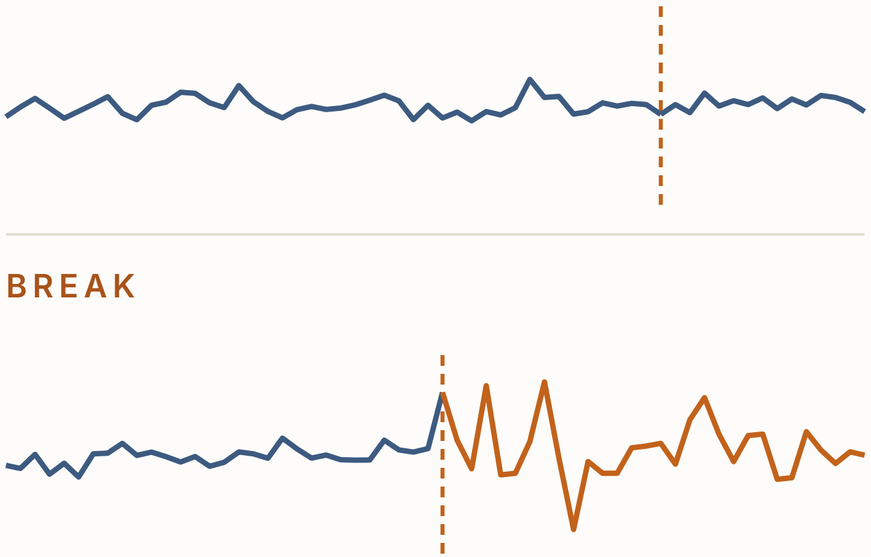
BACKGROUND

The task

Each example is a **time series**: a list of numbers recorded one after another, like a daily temperature or a machine's sensor reading.

Each one has a **marked point**. The question: **did the behaviour change at that point?** That change is called a **structural break**.

I get 24,000 practice series with the answer, and must give a probability for 4,000 new ones.



SCORING

How I am scored: AUC

Pick one series that has a break and one that doesn't. **AUC is how often my system gives the break series the higher probability.**



0.50
A coin flip

0.88
My best: right 88 times out of 100

1.00
Perfect every time

Two scores appear in this deck: **public** is Kaggle's score on the test series; **local** is my own check on the practice series.

OVERVIEW

The four tries at a glance

		Public score (axis starts at 0.82)
try1	Compare the numbers before and after the point	0.833
try2	Also ask how the numbers were produced	0.875
try3	The full model: a large committee, then corrections	0.881 · best
try4	try3 cut down to fit the time limit	0.876 · final

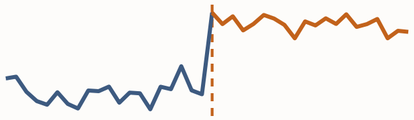
try3 is my best model, but it is too slow for the time limit. So I nerfed it myself: try4 is the nerfed version, and my final model.

TRY1 · THE IDEA



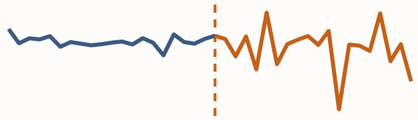
Measure both halves and compare

Cut each series at the marked point. Describe the “before” half and the “after” half with many measurements, then look at the differences.



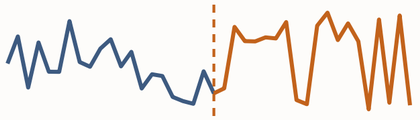
Level

Is the average higher or lower after the point?



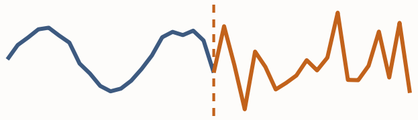
Spread

Do the values swing more widely?



Shape

Do the two halves look like they come from the same mix of values?



Memory

Does today’s value still predict tomorrow’s as well as before?

TRY1 · HOW IT DECIDES · 0.833

977 measurements, two models that vote

01

Features

977 numbers per series, each one a before-vs-after comparison like those on the last slide.

02

Model

A program that learns from the 24,000 answered examples. Mine are “decision trees”: thousands of learned yes/no questions, such as “did the spread double?”

03

Vote

Two tree programs (XGBoost and LightGBM) each rank the series; I average their rankings.

0.833 Most of my time went into try1. It also got the lowest score of the four. A solid start, I tell myself.

TRY2 · THE IDEA



Ask how the numbers were produced

try1 asked “do the halves look different?”. try2 also asks “**did the process making the numbers change?**” I fit simple descriptions of that process to each half and compare them.

01 Rhythm
a repeating cycle that changes speed or stops

02 Trend
a steady climb that bends or reverses

03 Jumps
sudden spikes appearing or vanishing

04 Calm vs stormy
quiet and noisy periods switching

05 Noise
the random part getting louder or changing type

06 Momentum
how strongly one value pulls the next

TRY2 · RESULT · 0.875

More and better measurements, one model

3,625 features per series: try1's comparisons, the same comparisons on windows of different sizes around the point, and the new process measurements.

Fed to a single tree model (LightGBM).

VS TRY1

+0.042

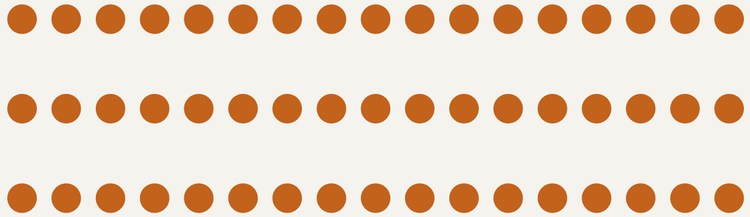
over try1, the biggest single jump of the project

Lesson: describing how the data was made beats generic statistics (and beats weeks of polishing try1).

TRY3 · THE FULL MODEL · 0.881

A committee of 51 experts

One model has blind spots. try3 trains **51 different models**, each with a different way of looking, and combines their opinions, like asking a panel instead of one person.



51 MODELS

About 5,100 measurements in total. Spoiler: one of these 51 ends up doing most of the work.

Tree models

Yes/no-question models, each given a different set of measurements

Neural networks

Models loosely inspired by the brain, good at blending many weak signals

Sequence readers

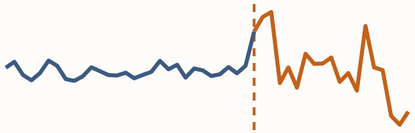
Networks that read the raw numbers directly, with no hand-made measurements

TRY3 · INGREDIENT 1



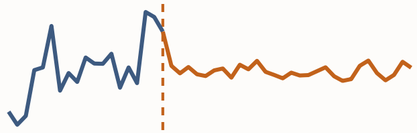
Look at each series 8 ways

A break is still a break if you play the series backwards or upside down. So I make extra copies of every series and let models learn from all of them.



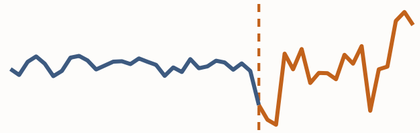
Original

As recorded



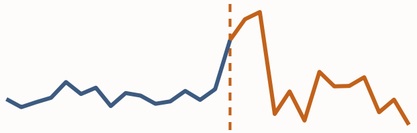
Reversed

Played backwards



Flipped

Upside down, and both



4 crops

Zoomed in around the point

When scoring, these models answer for every copy and I average, like asking someone to judge a photo and its mirror image.

TRY3 · INGREDIENT 2

The specialist that carried the team

WHO

Specialist models

Models that see only the “how was it produced” measurements from try2, studied on the original and the zoomed-in copies.

WHAT THEY GET

Noise priors

A lookup learned from the practice answers: “when the random noise looks like this, how often was there a break?” Given to the specialists as a starting guess.

0.874

One specialist, on its own, scores this: almost as high as the whole 51-model committee. The other 50 were mostly moral support.

TRY3 · INGREDIENT 3

Combine, then fine-tune the answer



1. **Blend**: combine the 51 opinions into one, giving more weight to the experts that proved more reliable.

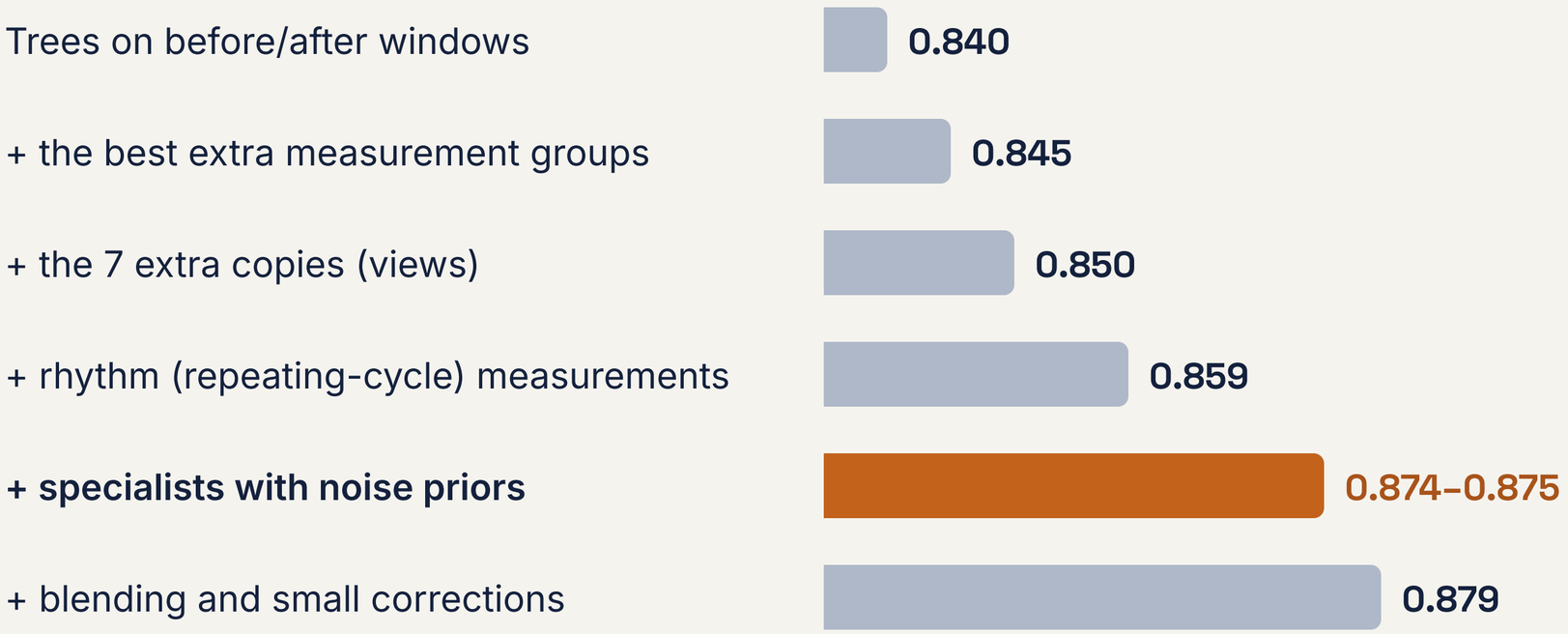
2. **Small corrections**: dozens of small adjustment steps, each nudging the blended answer using one extra signal. A step is kept only if it improves the score on data it didn’t learn from.

3. **Practice on made-up data**: one more neural network, trained on series I generated myself (so the answers are known), adds a final correction. This took me from 0.879 to 0.881.

TRY3 · WHAT EACH PART ADDED

What each ingredient was worth

Local score as each part was added (axis starts at 0.835)



BIGGEST GAIN

+0.015

from the specialists: the biggest jump on the ladder. One idea beat weeks of tinkering.

TRY3 · THE CATCH

The best score is expensive to run

473

separate steps to train it, many of which must wait for the one before

~1 day

of computer time to retrain it from scratch, even with two graphics cards

hours

to score new series. The rules allow 30 minutes on 2 processor cores with 8 GB of memory

So my best model could not be my final submission. Time to nerf my own model: how much of it do I actually need?

try3, nerfed to fit the rules

THE RULE, FROM SCRATCH:

30 min
to score 4,000 new series

2 cores
of processor

8 GB
of memory

The goal: nerf it as gently as possible, so it still fits the rule. Three kinds of cuts:

- **Cut 1 · fewer models**
- **Cut 2 · fewer measurements**
- **Cut 3 · less wasted computing**

CUT 1 · THE COMMITTEE

From 51 experts to 3

TRY3



51 models of five kinds, including slow neural networks and sequence readers, then dozens of correction steps and a made-up-data network.

TRY4



Only fast tree models: 2 LightGBM and 1 XGBoost. Their rankings are averaged. No neural networks and no correction steps.

Tree models do most of the work and run in seconds. The other 48 cost hours for the last fraction of a point, so they were let go.

CUT 2 · THE MEASUREMENTS

From about 5,100 measurements to 136

100

I ranked every measurement by how much it helps and kept only the **top 100**. The rest add very little once the best ones are in.

+36

I kept the **noise priors** from try3's specialists, its most valuable ingredient.

−4

I dropped the **4 slowest groups** of measurements entirely: about 7 minutes saved for a loss of just 0.0004. A bargain.

CUT 3 · WASTED COMPUTING

Only calculate what gets used

01

Skip unused columns

Each measurement group computes dozens of numbers at once. try4's versions compute only the ones the models read, with identical results.

02

Fewer steps

77 measurement steps instead of try3's 473 training steps, because only the needed groups are built.

03

Work in parallel

Steps that don't depend on each other run at the same time on all available processor cores.

I checked that the trimmed calculations give exactly the same numbers as the full ones.

DECISION

The trade-off I chose

TRY3 · BEST SCORE, BENCHED

0.881

51 models plus corrections. About a day to train, hours to score new series: breaks the 30-minute rule.

TRY4 · NERFED, AND FINAL

0.876

3 tree models, 136 measurements. About 22 minutes on 2 cores with 2.6 GB of memory. Keeps **99.4%** of try3's score.

